

ML Intern × ThingsBoard — Engineering Executive Report

Audience: Engineering Team — Build Immediately
Context: 1000+ Bank of India branch devices · Private Cloud ThingsBoard · Dahua NVR · Hikvision Cameras · Multi-system telemetry (CCTV, FAS, IAS, ACS, BAS, TLS)
Date: May 2026

Table of Contents

- 1. [System Context — What We're Working With](#)
- 2. [ML Intern — Current Functionality](#)
- 3. [Where ML Intern Plugs Into Our Stack](#)
- 4. [Real-World Use Cases — Our Security Domain](#)
- 5. [Development Guide — How to Build This](#)
- 6. [Gap Analysis — What Cannot Be Done Yet](#)
- 7. [Data Pipeline — ThingsBoard API → Training Data](#)
- 8. [Toolchain Map — Who Does What](#)
- 9. [Risk Register](#)
- 10. [Immediate Next Steps](#)

1. System Context

What We Have Right Now

Layer	Component	Status
Device telemetry	1000+ branch JSON docs via ThingsBoard REST API	✓ Available
ThingsBoard	Private cloud deployment	✓ Running
Data fields	usage_history, ticketStatus (21 flags), systemHealth, camera_status, disk, cpu, battery_voltage, BASheartbeatCount	✓ Rich
Labelled fault data	None — raw unlabelled JSON only	✗ Missing
AWS self-hosted TB	Planned purchase	⌚ Pending
JetBrains (DataSpell, WebStorm)	Licensed	✓ Available
Claude API / AI tools	Subscribed	✓ Available

What the Telemetry Data Looks Like (1 branch)

```
{
  "branchName": "BRANCH SERAMPORE",
  "branchId": 4252,
  "nbgName": "NBG EAST",
  "usage_history": [
    { "date": "2025-10-16", "count": 14.6 },
  ]
}
```

```

    { "date": "2025-10-17", "count": 52.3 },
    ...
    { "date": "2025-11-28", "count": 0 },
    { "date": "2026-04-24", "count": 0 }
  ],
  "ticketStatus": {
    "CAM_DIS": false,
    "HDD_ERROR": false,
    "FAS_OFF": false,
    "ACS_OFF": true,
    "POWER_OFF": false,
    "BATTERY_LOW": false
    ...
  },
  "systemHealth": {
    "disk": 76.7,
    "cpu": 2.5,
    "battery_voltage": 14.0,
    "net_sent_mb": 407
  }
}

```

Key finding already visible in data: Branch Serampore shows `usage = 0` for 130+ consecutive days (Nov 28, 2025 → Apr 2026). This is the primary "fault gap" pattern the CEO wants to detect — at scale across all 1000+ branches.

2. ML Intern — Current Functionality

Repo: [huggingface/ml-intern](#) (open-sourced April 2026)

What it is: An autonomous AI agent that runs end-to-end ML training workflows — literature review → dataset prep → script generation → GPU job submission → evaluation → iteration — with a human approval gate before any paid compute runs.

What It Can Do Right Now

✅ Autonomous Literature Research

- Searches arXiv and Hugging Face Papers database
- Reads full papers, walks citation graphs
- Selects the best training methodology without human guidance
- **For us:** Given "fine-tune a model to predict device faults from IoT telemetry", it finds the right architecture (likely lightweight transformer + time-series encoder) without us specifying it

✅ Dataset Preparation & Formatting

- Reads uploaded data (CSV, JSON, Parquet)
- Detects schema inconsistencies and mismatched field names
- Reformats data into the training schema required by the base model
- Filters out malformed or low-quality records
- **For us:** Normalises the inconsistent field names across 1000 branch JSON docs before training begins

✅ End-to-End Training Script Generation

- Writes complete runnable Python scripts using HuggingFace `transformers` + `trl` libraries
- Handles tokenisation, dataset splitting, hyperparameter config, evaluation loops
- **For us:** No in-house ML engineer needed to write training code

✔ **Cloud GPU Management via `hf_jobs`**

- Submits training jobs to Hugging Face cloud infrastructure (A100 GPUs)
- Monitors job progress, reads eval outputs
- Resubmits with adjusted hyperparameters if results are poor
- **Hard constraint:** Every paid GPU job requires explicit human approval before submission — no surprise bills

✔ **Iterative Evaluation & Failure Diagnosis**

- Reads evaluation metrics after each run
- Diagnoses failures (loss not converging, overfitting, data leakage)
- Proposes and runs ablations automatically
- **Published benchmark:** Improved Qwen3-1.7B scientific reasoning from 8.5% → 32% on GPQA in 10 hours of autonomous operation

✔ **Versioning & Audit Trail**

- Every training run, checkpoint, and session log is automatically pushed to a private HuggingFace Hub repository
- Full reproducibility — can show exactly what data trained any given model version

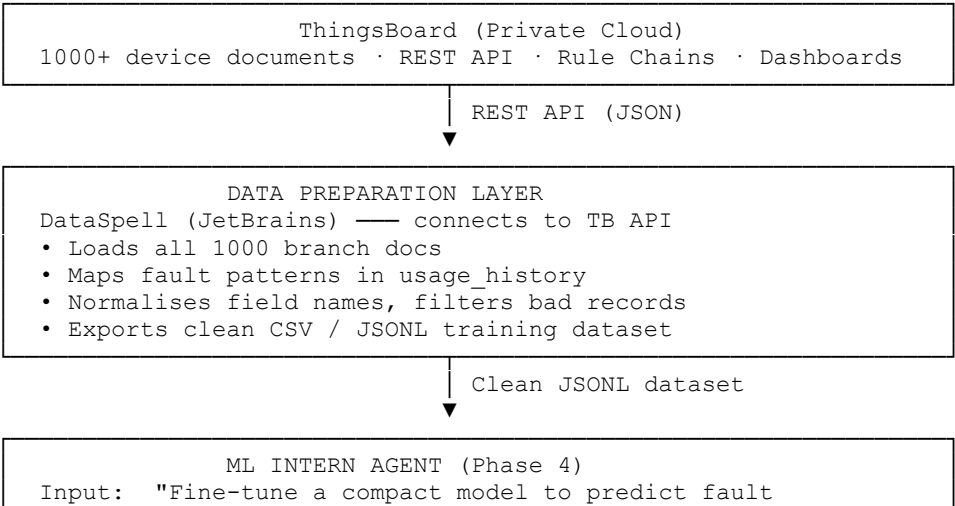
✔ **Local / Private Execution**

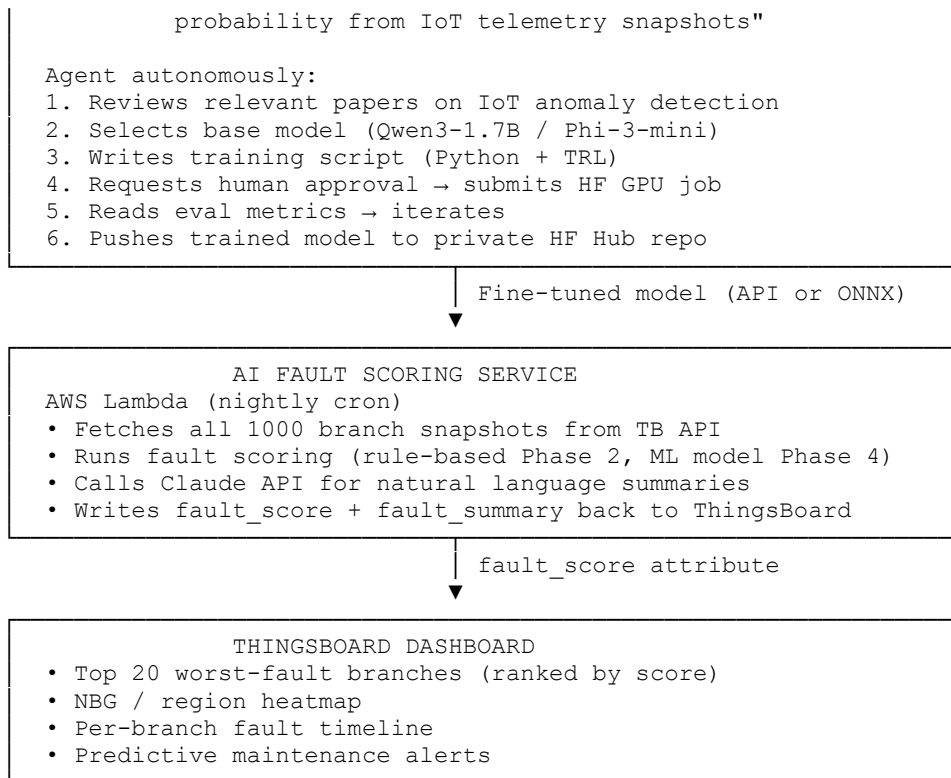
- Can run entirely on local hardware using Ollama, vLLM, or LM Studio as the orchestration LLM
- Proprietary telemetry data never leaves our infrastructure during the planning phase

Hard Limits of the Agentic Loop

Constraint	Value
Max iterations per task	300
Orchestration LLM required	Claude (Anthropic) or OpenAI API key
GPU compute	HuggingFace cloud (A100) or local hardware
Tool maturity	Released April 2026 — expect rough edges
Context window per session	Finite — complex pipelines must be broken into sequential tasks

3. Where ML Intern Plugs Into Our Stack





4. Real-World Use Cases

Use Case 1 — Fault Gap Detection (Highest Priority, Build First)

Problem: Branch Serampore was dead for 130+ days. Nobody noticed.

Solution: Nightly Lambda scans `usage_history` across all 1000 docs for zero-streaks ≥ 3 days. Ranks by severity. Pushes a `gap_score` to ThingsBoard.

ML Intern role: None required for Phase 1. Pure rule-based logic. ML Intern enters in Phase 4 to predict the gap *before* it happens.

Output: ThingsBoard widget showing "Top 20 branches with active fault gaps" updated every morning at 6 AM.

Use Case 2 — Multi-Flag Ticket Scoring

Problem: 21 binary `ticketStatus` flags per device. No single flag tells the full story. A device with `ACS_OFF` + `CAM_DIS` + `disk > 80%` is far more critical than one with just `ACS_OFF`.

Solution: Weighted fault score formula:

```
def compute_fault_score(device: dict) -> float:
    score = 0.0
    ts = device.get("ticketStatus", {})
    sh = device.get("systemHealth", {})

    # Critical flags (weight 30)
    if ts.get("POWER_OFF"): score += 30
    if ts.get("HDD_ERROR"): score += 30
    if ts.get("CAM_DIS"): score += 25

    # Warning flags (weight 15)
    if ts.get("FAS_OFF"): score += 20 # fire alarm off at a bank = serious
    if ts.get("ACS_OFF"): score += 15
```

```

if ts.get("IAS_FAULT"):      score += 15
if ts.get("BATTERY_LOW"):    score += 10

# Resource thresholds (weight 10)
if sh.get("disk", 0) > 85:    score += 15
if sh.get("disk", 0) > 75:    score += 8
if sh.get("cpu", 0) > 90:     score += 10

# Usage gap (weight 40 – highest)
gap_days = compute_gap_days(device.get("usage_history", []))
if gap_days >= 30:           score += 40
elif gap_days >= 7:          score += 25
elif gap_days >= 3:          score += 10

return round(min(score, 100), 2)

```

ML Intern role: Once 3–6 months of scored data accumulates and engineers label outcomes ("this score-80 branch needed physical visit"), ML Intern fine-tunes the weight model to better reflect real-world severity.

Use Case 3 — Natural Language Fault Summary (Claude API)

Problem: Ops engineers read raw JSON. Branch managers cannot.

Solution: Claude API call per device, once per night, generating a plain-English summary.

```

import anthropic

def generate_fault_summary(device_snapshot: dict) -> str:
    client = anthropic.Anthropic()
    prompt = f"""
You are an IoT fault analyst for a CCTV and security systems company.
Analyse this device snapshot and write a 2-3 sentence plain English summary
of its current health status and any recommended actions.
Be specific about device model, fault type, and urgency.

Device data:
{json.dumps(device_snapshot, indent=2)}
"""
    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=300,
        messages=[{"role": "user", "content": prompt}]
    )
    return response.content[0].text

```

Output example:

"Branch Serampore (Bank of India, NBG East) has been offline for 132 days with usage count at zero since November 28, 2025. The Dahua ST7108 NVR shows no recent heartbeat. The Access Control System is also flagged as OFF. Immediate on-site engineer visit is recommended to restore connectivity and verify the NVR power supply."

Use Case 4 — Predictive Fault Model (ML Intern — Phase 4)

Problem: Fault gap detection is reactive. We want to know 7 days *before* a device goes offline.

Solution: Fine-tune a compact model (Qwen3-1.7B or Phi-3-mini) on our device telemetry history. Input: last 30-day rolling window of disk%, cpu%, net_sent_mb, BASheartbeatCount, battery_voltage, ticketStatus flag counts.

Output: fault_probability_7d (0.0 → 1.0).

ML Intern prompt (Phase 4 input):

Fine-tune a compact classification model for binary IoT fault prediction.

TASK: Given a 30-day rolling window of device telemetry features (disk%, cpu%, heartbeat_count, net_sent_mb, battery_voltage, active_ticket_flags), predict whether the device will go into a fault state (usage = 0 for 3+ consecutive days) within the next 7 days.

BASE MODEL: Qwen3-1.7B (smallest feasible option for edge deployment later)
TRAINING DATA: [attached JSONL – 1000 branches × 12 months = ~365,000 rows]
EVALUATION METRIC: F1 score on fault class (minority class – handle imbalance)
METHODOLOGY: Search for best approach – consider tabular transformers, time-series encoders, or instruction-tuned classification.

OUTPUT: Push final model to private HF repo: [our-org]/device-fault-predictor

Use Case 5 — Camera Health Tracker

Problem: CH3 on Serampore has no camera_status field. Across 1000 branches this pattern is invisible without automated scanning.

Solution: Script that scans all branch docs, flags channels with missing camera_status, channels where total_recording_days = 0, and channels that have been in Disconnected state for > 48h.

ML Intern role: None needed. This is pure data quality alerting. Built in DataSpell or as a Lambda function.

Use Case 6 — Region/NBG Fault Heatmap

Problem: CEO wants macro visibility — which region is worst right now?

Solution: Group all 1000+ branches by nbgName, compute average fault_score per region, render in ThingsBoard as a geographic heatmap widget.

No ML Intern needed. Pure aggregation query:

```
from collections import defaultdict

def compute_regional_scores(all_branches: list) -> dict:
    region_scores = defaultdict(list)
    for branch in all_branches:
        nbg = branch.get("nbgName", "UNKNOWN")
        score = compute_fault_score(branch)
        region_scores[nbg].append(score)
    return {
        region: round(sum(scores) / len(scores), 1)
        for region, scores in region_scores.items()
    }
```

5. Development Guide

Phase 1 — Data Audit (Week 1–2) — DataSpell

Goal: Understand what we actually have before writing a single line of production code.

```
# DataSpell notebook – connect to ThingsBoard REST API
import requests
import pandas as pd
import json
```

```
TB_URL = "https://your-tb-instance.com"
TB_TOKEN = "Bearer YOUR_JWT_TOKEN"
```

```
def fetch_all_devices():
    headers = {"X-Authorization": TB_TOKEN}
    # Paginate through all device attributes
    devices = []
    page = 0
    while True:
        resp = requests.get(
            f"{TB_URL}/api/devices?pageSize=100&page={page}",
            headers=headers
        )
        data = resp.json()
        devices.extend(data["data"])
        if not data.get("hasNext"):
            break
        page += 1
    return devices

# Load into DataFrame and profile
df = pd.DataFrame(fetch_all_devices())

# Key audit checks
print(f"Total branches: {len(df)}")
print(f"Branches with usage_history: {df['usage_history'].notna().sum()}")
print(f"Branches with ACS_OFF: {df['ticketStatus'].apply(lambda x: x.get('ACS_OFF', False) if x else False).sum()}")
```

Deliverable: A data quality scorecard — per branch, per field, completeness %, gap frequency.

Phase 2 — Rule-Based Alerting (Week 2–3) — WebStorm + ThingsBoard

Goal: Immediate value to ops team using zero ML.

1. Implement `compute_fault_score()` as a Node.js Lambda
2. Schedule nightly via AWS EventBridge (or equivalent in private cloud)
3. Write `fault_score` back to ThingsBoard device attributes via REST API
4. Configure ThingsBoard alarm rule: `fault_score > 70` → trigger alert
5. Build ThingsBoard dashboard widget: ranked list of top 20 fault branches

```
// AWS Lambda - nightly fault scorer (Node.js)
const TB_API = process.env.TB_API_URL;
const TB_TOKEN = process.env.TB_TOKEN;
const ANTHROPIC_KEY = process.env.ANTHROPIC_API_KEY;

exports.handler = async () => {
    const devices = await fetchAllDevicesFromTB();

    for (const device of devices) {
        const score = computeFaultScore(device);
        const summary = score > 50
            ? await generateAISummary(device, ANTHROPIC_KEY)
            : "Device operating normally.";

        await writeFaultScoreToTB(device.id, { fault_score: score, fault_summary: summary });
    }

    console.log(`Scored ${devices.length} branches.`);
};
```

Phase 3 — AI Natural Language Layer (Week 3–4) — Claude API

Goal: Human-readable fault summaries for branch managers and ops team.

- Only call Claude API for devices with `fault_score > 50` (cost control)
- Cache summaries for 24h — do not regenerate if device state hasn't changed
- Write `fault_summary` as a ThingsBoard device attribute
- Surface in ThingsBoard dashboard as a text widget per device

Cost estimate: At ~\$0.003 per summary (Claude Haiku tier), 200 high-fault devices/night = ~\$0.60/night = ~\$18/month. Negligible.

Phase 4 — ML Intern Fine-Tuning (Month 2+)

Prerequisites before running ML Intern:

Prerequisite	How to Get It	Owner
Labelled fault dataset	Run Phases 1–3 for 4–8 weeks, then have engineers label which scored events were genuine faults vs false positives	Engineering
JSONL training file	DataSpell script exports ThingsBoard API data into training format	Engineering
HuggingFace account + Pro tier	Register at huggingface.co, upgrade for GPU access	Engineering/Finance
Anthropic API key	Already subscribed — use existing key as ML Intern's orchestration brain	Engineering
ML Intern installed	<code>pip install ml-intern</code> or clone repo	Engineering

Running ML Intern:

```
# Install
pip install ml-intern

# Set environment
export ANTHROPIC_API_KEY="sk-ant-..."
export HF_TOKEN="hf_..."

# Launch with our task
ml-intern run \
  --task "Fine-tune a compact fault prediction model on CCTV device telemetry.
        Dataset attached. Predict fault_probability_7d from 30-day feature windows.
        Use F1 on fault class as primary metric. Push to hf://our-org/device-fault-predictor" \
  --dataset ./training_data/fault_dataset.jsonl \
  --approve-compute # Enables interactive approval before GPU spend
```

6. Gap Analysis

What ML Intern Cannot Do for Our Use Case

Gap	Why	Workaround
Cannot label our data	ML Intern trains on labelled data — it cannot generate ground truth fault labels from raw telemetry	Engineers must label fault events manually for 4–8 weeks before training is viable
Cannot connect to ThingsBoard API directly	ML Intern reads local files or HuggingFace datasets — it has no native TB integration	DataSpell extracts data from TB API → exports JSONL → ML Intern reads the file

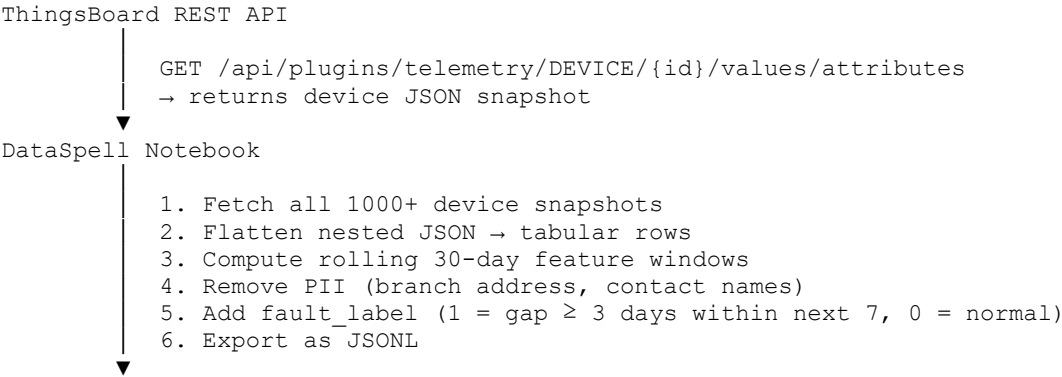
Gap	Why	Workaround
Cannot run in real-time	The agentic loop is designed for training workflows, not live inference	Trained model is deployed as a separate inference service; ML Intern is only used for training
Cannot explain root cause	Predicts fault probability from patterns — cannot say "this happened because of a power cut" without labelled root-cause data	Phase 4B: after Phase 4A runs 6 months, add root-cause labels to training data
Cannot fix missing data gaps	The Nov 2–27 data hole in usage_history is a pipeline bug — ML Intern cannot reconstruct it	Fix the ThingsBoard data ingestion pipeline before training. Impute or drop missing windows.
Cannot deploy the model	Pushes to HuggingFace Hub only — does not integrate into ThingsBoard or AWS Lambda	Engineering deploys the HF model as an ONNX file or via HF Inference API inside the Lambda
Cannot run 1000 devices in parallel	Each ML Intern session has a 300-iteration cap	Fine-tuning is a one-time (or periodic) batch job — this limit doesn't affect inference
Cannot flag regulatory risk	Has no awareness of GDPR or banking sector data obligations	Legal/DPO review required before any data leaves the private cloud for HF GPU training

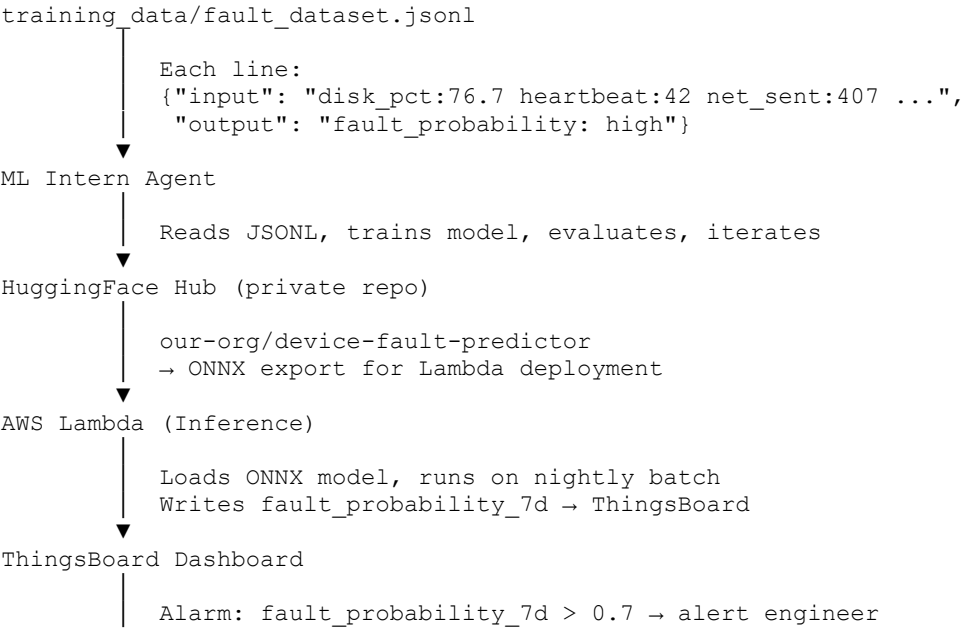
What ThingsBoard Currently Cannot Do (Gaps Addressed by Our Build)

Gap	Current State	Our Solution
Fault gap detection	None — nobody knows a branch is dead for 130 days	Phase 1 — nightly Lambda + <code>fault_score</code> attribute
Predictive alerts	Reactive only — alerts fire after fault, not before	Phase 4 — ML Intern trained predictive model
Natural language summaries	Raw JSON/numbers only	Phase 3 — Claude API summaries written to TB
Regional fault heatmap	No aggregation across NBG regions	Phase 2 — custom aggregation widget
Camera channel health	No automated per-channel health check	Phase 1 — DataSpell audit script
SD card recording audit	No visibility into <code>total_recording_days = 0</code> pattern	Phase 1 — included in data audit

7. Data Pipeline — ThingsBoard API → Training Data

Step-by-Step: From Raw TB Data to ML Intern Input





8. Toolchain Map — Who Does What

Tool	Phase	Role	Effort to Set Up
DataSpell	1, 4	Data exploration, audit, JSONL export, TB API connection	Low — already licensed
WebStorm	2, 3	Lambda code, TB rule chain config, API integration	Low — already licensed
Claude API	3	Natural language fault summaries	Low — already subscribed
ThingsBoard	2, 3, 4	Dashboard, rule chains, alarms, attribute storage	Medium — rule chain config
AWS Lambda	2, 3, 4	Nightly scoring orchestrator	Medium — infra provisioning
ML Intern	4 only	Autonomous model training	Medium — requires labelled data first
HuggingFace Hub	4	Model versioning, ONNX export	Low — free tier works
AWS EventBridge	2+	Nightly cron scheduler	Low
AWS SNS/SES	2+	Email/SMS alerts to engineers	Low

9. Risk Register

Risk	Severity	Likelihood	Mitigation
No labelled fault data → ML Intern useless in Phase 4	High	Certain until addressed	Run Phases 1–3 first; engineer labelling starts in parallel
TB private cloud → no direct DB access	Medium	High	All access via REST API — slower but workable
Data privacy: telemetry leaving private cloud for HF GPU	High	Medium	Either anonymise fully before export, OR run ML Intern with local GPU/vLLM

Risk	Severity	Likelihood	Mitigation
Nov 2–27 data gap in usage_history	Medium	Already present	Identify source of pipeline failure; impute with rolling mean for training
ML Intern tool immaturity (released April 2026)	Medium	High	Treat Phase 4 as a time-boxed experiment; do not block Phases 1–3 on it
ThingsBoard AWS migration mid-project	High	Planned	Build the Lambda + AI layer to be TB-instance-agnostic (API calls only, no direct DB)
CH3 camera missing status field across N branches	Low	Unknown	Phase 1 audit quantifies scale; fix is firmware/provisioning, not software

10. Immediate Next Steps

Ordered for immediate engineering action. No waiting, no procurement needed for Steps 1–3.

This Week

- [] **Step 1:** Open DataSpell. Connect to ThingsBoard REST API. Write a notebook that fetches all 1000+ device documents. Count: how many branches have `usage_history`? How many have `usage = 0` right now? How many have `ACS_OFF = true`? This is your baseline audit. **Time: 1–2 days.**
- [] **Step 2:** Implement `compute_fault_score()` function. Run it across all fetched devices. Export a CSV: `branch_name, nbg, fault_score, top_fault_reason`. Share with CEO. This is the first concrete output. **Time: 1 day.**
- [] **Step 3:** Identify the data gap — why does Serampore's `usage_history` jump from Nov 2 to Nov 28 with no data? Is this a ThingsBoard ingestion bug? A device-side issue? Document the finding. **Time: half a day.**

Next Two Weeks

- [] **Step 4:** Write the AWS Lambda nightly scorer. Deploys `compute_fault_score()` + Claude API summary generation. Writes `fault_score` and `fault_summary` back to ThingsBoard device attributes. Schedule via EventBridge. **Time: 3–4 days.**
- [] **Step 5:** Build ThingsBoard dashboard: "Top 20 Fault Branches" widget + regional NBG heatmap. Use `fault_score` attribute as the data source. **Time: 2 days.**

Month 2

- [] **Step 6:** Engineers begin labelling fault events in a shared spreadsheet — for each high-score branch that required a physical visit, record `fault_type` (power, camera, NVR, network) and `root_cause`. This is your Phase 4 training data.
- [] **Step 7:** After 4–8 weeks of labelled data accumulates, run DataSpell to generate the JSONL training file. Verify data quality. Then invoke ML Intern with the fine-tuning prompt. Budget: ~£100–150 in HuggingFace GPU credits for the first run.

Summary

Phase	What Gets Built	ML Intern Involved?	Timeline
1 — Audit	Data quality scorecard, gap mapping	No	Week 1–2
2 — Rule Alerts	Fault scorer Lambda, TB dashboard	No	Week 2–3
3 — AI Summaries	Claude API natural language summaries	No	Week 3–4
4 — Predictive Model	Fine-tuned fault predictor, 7-day ahead alerts	Yes — this is its role	Month 2+

ML Intern is not a Phase 1 tool. It is a Phase 4 tool.
 The first three phases build the labelled dataset that makes Phase 4 possible and worthwhile.
 Without them, ML Intern has nothing meaningful to train on.
